

Improving declustering for Qcify AIR

Realization Document

Julia Drygalska
Student Bachelor Applied Computer Science

Table of Contents

1. INTRODUCTION	2
1.1. Internship Assignment	3
2. ANALYSIS	4
2.1. Previous Experimental Approach	5
2.2. Research on Segmentation Strategies	5
2.3. Dataset and Target Representation Analysis	6
2.4. Evaluation Criteria	8
2.5. Selected Direction	8
3. IMPLEMENTATION	9
3.1. Main ESPNetV2 Training Setup	10
3.2. Boundary and Distance Map Integration in Training	10
3.3. Loss Functions and Penalty Terms	11
3.4. Training Procedure	11
3.5. Additional FBSNet Experiment	12
4. EVALUATION AND RESULTS	12
4.1. Evaluation Environment	13
4.2. Quantitative Evaluation Method	13
4.3. Selected Model Results	14
4.4. Runtime Results	15
4.5. Visual Evaluation	16
4.6. Discussion of Results	26
5. CONCLUSION	26
5.1. Future Work	27
REFERENCE LIST	27
ATTACHMENTS	29

1. Introduction

This realisation document describes the execution of the internship project carried out at Qcify. The project's focus was on improving the segmentation behaviour of the Qcify AIR machine in cases where multiple almonds appear close together in the image.

Qcify AIR is an industrial optical sorting machine used in inline food processing. The system uses image-based inspection and artificial intelligence to analyse products moving through the machine and support automated sorting decisions. In this context, accurate segmentation is important because the machine has to recognize product regions reliably and process them at high speed.

The main problem investigated during this project was declustering. In this context, declustering refers to the separation of touching or closely positioned nuts in the segmentation output. In previous experiments, some closely positioned nuts could be predicted as one connected object instead of multiple individual objects. This problem could reduce the usefulness of the segmentation result, especially for object counting, defect localisation and later sorting decisions.

The assignment was approached as a research and development project. The main goal of this project was not to replace the current vision pipeline, but to investigate how the previous experimental approach could be improved for this problem. To do that, several experimental directions were explored, including the use of boundary masks, distance maps, loss functions and additional penalty terms. The following chapters describe the analysis, implementation, evaluation method, results and conclusions of the project.

1.1. Internship Assignment

The internship assignment consisted of investigating possible improvements for almond declustering in the Qcify AIR vision pipeline. At the start of the project, there was no declustering model actively used in production for this purpose. There had been previous experimental attempts, but they did not provide reliable enough results to be used in the machine.

The main limitation that was observed in earlier experiments was that stronger declustering could remove too much useful information from nut regions. In some cases, this created holes inside the masks or cut away parts of the object. This could negatively affect later classification steps, since the classifier still needs enough information about the object to decide whether it is good or defective.

The objective of this project was therefore not simply to separate touching objects as strongly as possible, but the goal was to investigate whether the segmentation target could be made more informative, so that the model could learn object boundaries while still preserving the shape and the content of the nut regions.

During the internship, several experimental setups were prepared and evaluated. The work included analysing previous experiments, researching new approaches, creating new datasets with additional training targets such as boundary masks and distance maps, training models with different input resolutions, and comparing the results using both quantitative metrics and visual inspection. The experiments mainly focused on almond images that were captured from the Qcify AIR machine.

The outcome of this assignment is an evaluated set of model configurations for improved declustering. The results were analysed not only in terms of pixel-level segmentation quality, but also in terms of object separation and runtime performance, since the future model is intended for use in a real-time industrial environment.

2. Analysis

Before implementing new model configurations, an analysis phase was conducted to understand the existing segmentation pipeline, available data and possible technical approaches that could improve declustering. The goal of this phase was to choose a direction that could improve the segmentation model while remaining compatible with the runtime constraints of the Qcify AIR machine.

The project was not only focused on achieving the highest possible segmentation metric. Since the model is intended to be used in an industrial optical sorting machine, the solution also had to remain efficient and practical to integrate into the existing pipeline. Because of that, the analysis included the previous experimental approach, possible segmentation strategies, available datasets, model architecture choices and evaluation criteria.

2.1. Previous Experimental Approach

The starting point of the project was previous experimental work on declustering, not a production segmentation model. At the time the project started, declustering segmentation models were not used in production, because the results were not sufficient for practical use.

The earlier approach can be understood as a semantic segmentation problem. In semantic segmentation, each pixel is assigned to a class. This is efficient because the model only needs to predict a class label for each pixel, instead of detecting and tracking separate object instances. However, this approach also creates a limitation for declustering. If all almond pixels belong to the same class, two touching objects can easily become one connected predicted region.

Therefore, semantic segmentation can be fast and suitable for real-time use, but it does not automatically know that two neighbouring nuts should be treated as separate instances. The model needs additional information to separate objects that belong to the same class.

One of the problems that was observed in earlier experiments was that stronger separation could remove too much information from segmented product regions. In some cases, this created unwanted holes inside the masks or cut away parts of the object.

This showed that the problem was not only to separate touching almonds, but to do so without damaging useful object information. A successful declustering approach should create clearer separation between neighbouring objects while preserving the shape of each nut as much as possible.

Because of this, the project was approached as a research and development task. The goal was to investigate whether the model could be improved in a controlled way using better target representations, boundary information, distance maps and adjusted loss functions, without introducing the same problems that made earlier experiments unsuitable for production.

2.2. Research on Segmentation Strategies

After analysing the limitation of earlier declustering attempts, different segmentation strategies were considered. The existing pipeline was based on semantic segmentation, where each pixel is assigned to a class. This approach is efficient, and it fits well into the current system, but it does not automatically differentiate between separate object instances when they belong to the same class.

As a possible alternative, instance segmentation was considered. This type of segmentation predicts separate masks for individual objects, which conceptually makes it suitable for the declustering problem. If each almond could be predicted as a separate instance, it would be easier to separate them at the output level.

Panoptic segmentation was also reviewed, because it combines semantic segmentation with instance-level separation. This approach, in theory, could provide both class information and object-level separation, which is relevant for a system that needs to understand individual almonds in the image.

However, instance and panoptic segmentation were not chosen as the main direction of this project. The main reason behind this decision is their expected runtime cost. Instance segmentation usually requires the model to predict separate masks for each detected object. Panoptic segmentation combines semantic and instance-level predictions. Both approaches can provide more direct object separation, but they also require a more complex output format, different evaluation logic and more processing time. Since Qcify AIR operates in a real-time environment, this made them less practical for the main experimental direction.

During the research phase, lightweight segmentation architectures were also reviewed. Lightweight segmentation architectures are models designed to reduce computational cost while still producing dense pixel-wise predictions. They usually use efficient convolution blocks, reduced channel sizes or simplified decoder structures to keep inference time low.

Most experiments were performed using ESPNetV2, because it was already available in the existing training pipeline and was suitable for efficient semantic segmentation. ESPNetV2 is a lightweight segmentation architecture that uses efficient convolutional blocks to reduce the number of operations while still extracting useful spatial features. In this project, ESPNetV2 tiny with a width multiplier of 0.5 was used. The width multiplier controls the number of channels that are used inside the network. A lower width multiplier makes the model smaller and faster, but it can also reduce the capacity of the model. In this project, a width multiplier of 0.5 was used to keep ESPNetV2 lightweight enough for runtime-sensitive experiments.

One additional architecture, FBSNet, was also investigated as an experimental direction. FBSNet is a lightweight encoder-decoder segmentation architecture. It was considered interesting because it includes mechanisms for preserving spatial detail, which can be useful for boundary-aware segmentation. Since declustering strongly depends on fine boundary regions between neighbouring objects, FBSNet was tested as an additional comparison.

FBSNet was not used as the main architecture for all experiments, it was implemented and tested in a limited number of runs to evaluate whether it could be useful for the declustering problem. The main experimental comparison remained focused on the ESPNetV2-based models.

A broader overview of lightweight semantic segmentation architectures reviewed during the research phase is included in Attachment B.

2.3. Dataset and Target Representation Analysis

The available data consisted of almond images captured from the Qcify AIR machine. The project focused on almonds, which made it possible to analyse the declustering problem for one specific product type and keep the experiments consistent.

The input width was fixed at 3072 pixels, because this corresponds to the horizontal image format used in the AIR pipeline. The tested input heights were 32 pixels and 64 pixels. The 32 px height is closer to the runtime-oriented setup used on the machine, while 64 px was tested to check whether additional vertical detail could improve declustering behaviour.

Four main training datasets were prepared for the experiments. Two datasets contained boundary masks, while the other two contained both boundary masks and distance maps. Each dataset type was prepared for two input heights: 32 pixels and 64 pixels.

Dataset type	Input size	Number of images
Boundaries	32 x 3072	28766
Boundaries	64 x 3072	13504
Distance maps + boundaries	32 x 3072	28766
Distance maps + boundaries	64 x 3072	13504

The number of images in the table refers to the number of sliced input images. Additional target files, such as boundary masks and distance maps, were generated for the corresponding images depending on the dataset type.

In all prepared datasets, the image width was kept at 3072 pixels, while the height was changed depending on the experiment. This resulted in input sizes of 32 x 3072 and 64 x 3072 pixels. The width remained without changes because it corresponds to the horizontal image format used in the Qcify AIR pipeline, while the height was varied to compare the effect of spatial detail on segmentation quality and runtime performance.

The dataset preparation process was based on an existing script for slicing images and adding boundaries and distance maps based on annotations. During the project, this workflow was extended and partially automated in a Jupyter Notebook to make the creation of new datasets faster and more consistent.

The notebook was designed to take the original images captured by the machine and creates initial annotations for them. This automatic annotation step was implemented using the API of the Segment Anything Model. SAM is an advanced AI model developed by Meta AI that can identify and segment any object in an image with minimal human input. Before including the images in the training data, these annotations were still checked afterwards and corrected if needed using CVAT. CVAT is a platform for managing and annotating visual datasets for AI and machine learning applications. This made it possible to speed up the annotation process while keeping manual verification as part of the workflow.

The notebook was also extended with the option to merge multiple datasets before slicing (since some of the available data already contained annotations). This was added to avoid manually combining JSON annotation files, which would be time-consuming and prone to mistakes.

After the preparation and optional merging steps, the existing slicing process was used to create the final training data. In this step the dataset is chosen together with the target image height, which is either 32 pixels or 64 pixels, and additional targets that should be generated: boundary masks, distance maps, or both. The script then slices the images and produces the final dataset structure used for model training.

The use of boundary masks was important because the original binary representation did not provide enough information about the structure of individual almonds. In a binary setup, the model only learns whether a pixel belongs to the background or an almond. For declustering, this is a limitation, because the model does not receive a direct signal about the border of two neighbouring nuts.

To address this, one of the main target representations used in this project divided the segmentation mask into three semantic classes: background, almond boundary and almond interior. This representation was selected since the boundary area is especially important for declustering. It can give the model more direct information about where one almond ends and the other one begins.

The boundary masks were generated with a fixed thickness of 8 pixels. This means that the boundary class covered an 8-pixel-wide region around the almond contour.

A boundary mask is a discrete target that marks the contour area of the almond. In this project, boundary pixels were treated as a separate semantic class, which was useful because the boundary region is the part of the mask that is most relevant for separating neighbouring almonds.

In later experiments distance maps were also investigated as an additional target. A distance map describes the internal structure of an object by representing distance-related information inside the almond region. Pixels closer to the centre of the almond region have a different value than pixels closer to the edge. This can help the model with understanding the internal structure of the object. In this project, distance maps were used as an additional supervision signal along with boundary masks, to support object separation while preserving the almond region itself.

The comparison between 32 px and 64 px inputs was included to analyse the trade-off between image detail and runtime performance. A higher input height can preserve more information about the object, but at the same time, it increases computational cost and can reduce inference speed, so the comparison had to consider both segmentation quality and runtime performance.

2.4. Evaluation Criteria

The evaluation criteria were defined during the analysis phase to make sure that the trained models would be compared from both a segmentation and a practical declustering perspective. Since the project focused on separating closely positioned almonds, it was not enough to evaluate the models with only standard pixel-level metrics.

Intersection over Union was used to measure the overlap between predicted masks and ground-truth masks. This metric is useful for checking the general segmentation quality, because it shows how well the predicted almond regions match the annotated regions. Both per-image IoU and dataset-level IoU were used. However, IoU alone is not enough to describe the quality of declustering. A model can achieve high IoU while still merging two neighbouring objects into one. In such a case, the pixel overlap may still be good, but the result is not correct from object-level perspective.

For that matter, additional metrics related to connected components and object counting were also included in the evaluation. Therefore, the evaluation also considered the predicted number of objects, object-count error, merge rate and split rate.

Runtime performance was another important measure. Because of that, the experiments also compared inference time and frames per second. A model with better segmentation quality would be of no use if it became too slow to be used on the machine.

Visual inspection was also used as part of evaluation. Some errors, such as holes inside almond masks, over-separation, or merged neighbouring almonds are easier to understand by looking at the predicted outputs. Because of that, visual examples were used to support final model comparison.

2.5. Selected Direction

The analysis phase led to a clear experimental direction of the project. The main focus was based on lightweight semantic segmentation, with ESPNetV2 used as primary architecture. This choice was based on the need to keep the model efficient enough for the real-time requirements of the Qcify AIR machine.

Instead of replacing the full segmentation approach, the experiments focused on improving the information provided to the model during training. The experiments were therefore built around boundary-aware target representations, distance maps, adjusted loss functions and additional penalty terms. These choices were

intended to improve the separation of neighbouring almonds while avoiding the issues that were observed in previous declustering attempts, where too much useful information was removed from the masks.

The experimental work was organised around a small number of main comparisons: 32 px versus 64 px input height, boundary-only target versus boundary and distance-maps targets, and different loss or penalty configurations. ESPNetV2 remained the main model architecture for these experiments. FBSNet was included only as an additional experimental architecture to determine whether a custom lightweight encoder-decoder model with spatial-detail preservation could be useful for this problem.

This direction made it possible to compare the trained models from multiple perspectives: segmentation quality, object-level declustering behaviour, runtime performance and visual output quality. These criteria were used later to determine which model configuration was the most suitable candidate for the Qcify AIR environment.

3. Implementation

The implementation phase focused on training and evaluating different model configurations based on the direction selected during the analysis phase. The datasets and target representations that were described in the previous chapter were used as input for the experiments. The goal was to test whether boundary-aware semantic segmentation, distance maps and adjusted loss functions could improve the separation of closely positioned almonds while keeping the model efficient.

Most experiments were performed using ESPNetV2 tiny with a width multiplier of 0.5. This architecture was already supported by the training pipeline. In addition to these experiments, one FBSNet model was implemented and trained as an additional comparison. The following sections describe the main model setup, integration of boundaries and distance maps, tested loss functions and the additional FBSNet experiment.

3.1. Main ESPNetV2 Training Setup

The main model used during the project was ESPNetV2 tiny. The original segmentation setup was extended from binary segmentation to a three-class semantic segmentation task. Instead of only predicting the background and object, the model was trained to predict three classes: background, almond boundary and almond interior. In the training target these three classes were represented as class 0 for background, class 1 for nut_boundary and class 2 for nut_interior.

The ESPNetV2 model output was extended depending on the experiment. In this context, a model head means a separate output branch of the neural network. Each head predicts a different type of information from the features extracted by the shared backbone. The semantic head produced the main three-class segmentation output: background, nut boundary and nut interior. The boundary head predicted a binary boundary mask, which provided additional supervision for object borders. The distance head predicted a distance map, which represented distance-related information inside the almond regions. Using multiple heads allowed the model to learn the main segmentation task together with additional structural information.

Most of the experiments conducted with ESPNetV2 compared different combinations of input resolution, target type and loss formulation. Some of the models were trained only with boundary information, and others used both boundary masks and distance maps. The experiments were performed using 32 x 3072 and 64 x 3072 input sizes to compare the trade-off between segmentation quality and runtime performance.

3.2. Boundary and Distance Map Integration in Training

Boundary information was integrated into the pipeline by converting the segmentation target into a three-class semantic mask. Pixels outside almond regions were assigned to the background class. Pixels that belonged to the boundary mask were assigned to the nut_boundary class, and the remaining pixels were assigned to the nut_interior class.

In addition to the semantic boundary class, several experiments also used a separate boundary head. It predicted the boundary mask independently from the semantic output. As a result, boundary information was learned in two ways: as a class in the semantic segmentation output and as an auxiliary binary boundary prediction.

Distance maps were included in later experiments as an additional training target. The distance head predicted a distance-related map, which was compared with the generated ground-truth distance map. A

distance map gives the model additional information about the internal structure of each almond. It was useful because it was meant to avoid removing too much useful information from the almond masks.

The implementation therefore included both boundary-only configurations and configurations that used both boundaries and distance maps.

3.3. Loss Functions and Penalty Terms

During the project, several loss formulations were implemented and tested. The first group of experiments used a combination of weighted Cross-Entropy Loss and weighted Dice Loss. Cross-Entropy loss was used for pixel-wise classification, and Dice Loss helped in improving region-level overlap between the predicted and ground-truth masks. In this setup the semantic loss combined 0.65 weighted Cross-Entropy with 0.35 weighted Dice Loss. Dice Loss was only calculated for the almond boundary and interior, as these classes take the least space of the image and it was important for the model to focus on them more, however the background class was still included in the Cross-Entropy term.

Later experiments used a combination of Cross-Entropy Loss and Focal Loss. This loss formulation combined 0.6 weighted Cross-Entropy with 0.4 weighted Focal Loss. Focal Loss was investigated because it gives more importance to difficult pixels and reduces the influence of pixels that are already classified correctly. This is relevant for declustering since boundary pixels and contact regions between neighbouring almonds are more difficult to classify than large background regions.

Class weighting was implemented to increase the importance of the boundary class. It was important since the boundary pixels take up only a small part of the image, but they are important for separating neighbouring almonds. In the later experiments, the priority template gave the highest priority to the boundary class, with values of 0.08 for background, 0.65 for almond boundary and 0.27 for almond interior. This encouraged the model to pay more attention to the boundary regions instead of simply ignoring them.

For experiments that used a boundary head, an additional boundary loss was added. It combined 0.6 BCEWithLogitsLoss with 0.4 Soft Dice Loss. The positive class weight for the boundary loss was calculated dynamically from each batch as the ratio between negative and positive pixels and was clamped to avoid unstable values.

For approaches that used distance maps, a distance loss was included. Early experiments used Smooth L1 Loss between the predicted distance output and the ground-truth distance map. Later experiments used a distance-weighted Smooth L1 Loss combined with Distance Gradient Loss. It was used to make the distance prediction more consistent with the structure of the ground-truth distance maps.

To reduce specific failure cases, additional penalty terms were introduced in later experiments. These penalties were used to penalize predictions such as almond interior on boundary pixels, interior on background pixels or boundaries inside interior regions. The purpose of these penalties was to improve object separation while avoiding overly aggressive cuts or holes inside almond masks.

The tested penalty terms included interior-on-boundary penalty, false positive distance penalty, interior-distance consistency, boundary-on-interior penalty, boundary-on-distance-center penalty and background-interior penalty. Not all of them were active in every experiment. They were introduced gradually to analyse how each change affected the quality of segmentation, declustering behaviour and visual output.

3.4. Training Procedure

The models were trained using the prepared datasets described in the analysis chapter. Each experiment used a specific combination of input height, target representation and loss configuration.

The validation process monitored IoU-based metrics, including per-image IoU and dataset-level IoU. The main checkpoint selection was based on validation per-image IoU. Early stopping was used to stop training when the validation metric did not improve for several epochs.

The optimizer used in the ESPNetV2 experiments was AdamW. The default learning rate was 0.0001, and the default weight decay was 0.0.

The models were trained on Qcify internal GPU servers. During the project, training jobs were executed on one of the three available AIP agents. All three machines ran Ubuntu 22.04 and were equipped with NVIDIA GeForce RTX 4090 GPUs. The exact server used for training depended on availability, so the models were not assigned to train on one specific machine, but all of them supported CUDA-accelerated training.

3.5. Additional FBSNet Experiment

One additional experiment was performed using FBSNet. This model was implemented as an extra experimental architecture to check whether a custom lightweight encoder-decoder network would be useful for this problem.

For the setup, this experiment used the same three-class semantic formulation as the other boundary-aware experiments. This allowed to compare the model against the ESPNetV2-based experiments using the same target representation.

The implemented FBSNet architecture included a convolutional stem, lightweight residual blocks, decoder upsampling blocks and a spatial detail path. The spatial detail path was applied to preserve local information from early feature maps, which could be valuable for detecting thin boundaries and contact regions between almonds. A spatial attention mechanism was used to highlight relevant local regions before combining them with the decoder output.

Because only a couple of experiments were run using this architecture, it was treated as an additional comparison rather than the main model direction. The main experimental comparison remained based on ESPNetV2.

A more detailed overview of the implemented FBSNet is included in Attachment A.

4. Evaluation and Results

The evaluation phase was used to compare the trained models from both quantitative and a visual perspective. Since the goal of this project was to improve declustering, the models could not be judged only by standard metrics. A useful model had to provide accurate masks, separate neighbouring almonds correctly, avoid unwanted holes or excessive cuts inside almond regions while remaining fast enough for the Qcify AIR machine.

Most of the evaluation was performed offline on a separate dataset. These datasets had the same structure as the training datasets but contained less data. This made the comparison between models more consistent since the same images were used for every model. Testing only on the physical machine would make a fair comparison more difficult because the almonds would be placed differently with every run. However, selected models were also tested on the AIR machine to visually inspect how the segmentation behaved in a real machine environment.

The evaluation workflow was collected into a Jupyter Notebook, so that the model metrics and visual outputs could be generated from one place. The notebook used dedicated scripts that were created for quantitative evaluation and visualisation. The metric script loads a selected model pipeline, runs inference on the evaluation dataset, calculates segmentation metrics and exports these results into a report. The visualisation script generates panels with input image, predicted mask, overlay and ground-truth masks. It also supports selecting samples based on count errors and exporting stitched full-image visualisations for better evaluation.

4.1. Evaluation Environment

Most model evaluation was performed on a laptop. This setup allowed to use the same dataset across all trained models, making the comparison controlled and repeatable.

Component	Specification
CPU	AMD Ryzen 9 6900HX with Radeon Graphics, 3.30 GHz
RAM	16 GB
GPU	NVIDIA GeForce RTX 3080 Ti Laptop GPU, 16 GB
Integrated GPU	AMD Radeon Graphics

Selected models were also tested on the computer used by the Qcify AIR machine. This was done mainly to see how the segmentation worked in the real environment and to understand runtime behaviour.

Component	Specification
CPU	Intel Core Ultra 9 285K
RAM	Kingston FURY 64 GB DDR5-5200
GPU	2 x INNO3D GeForce RTX 4080 SUPER X3

4.2. Quantitative Evaluation Method

The quantitative evaluation used both pixel-level and object-level metrics. Pixel-level metrics were necessary to measure how well the predicted masks matched the annotated masks. Object-level metrics were necessary because the project focused on declustering, where the number of separated objects is important.

The evaluation script calculated per-class metrics from the confusion matrix, including precision, recall, F1-score, Dice score, IoU and accuracy. For the three-class setup, the evaluated classes were background, nut boundary and nut interior.

For declustering, the script converted masks into foreground masks and counted connected components. This allowed the predicted number of objects to be compared with the ground-truth number of objects. Based on this comparison, the script calculated component count MAE, exact count accuracy, merge rate and split rate.

The most important metrics used in the comparison were:

Metric	Meaning
Nut IoU	Pixel-level overlap for the nut interior class
Count MAE	Average absolute error between predicted and ground-truth component count
Exact count accuracy	Percentage of images where the predicted object count matched the ground-truth
Merge rate	Percentage of images where the model predicted too few components
Split rate	Percentage of images where the model predicted too many components
FPS	Inference speed in frames per second

4.3. Selected Model Results

The full evaluation contained all ESPNetV2 runs for both 32 x 3072 and 64 x 3072 inputs. To keep the main document readable, only the most relevant configurations are shown in the table below.

Model/Run	Input size	Architecture	Main Loss and additional terms	Nut IoU	Count MAE	Exact count accuracy	Merge rate	Split rate
Test-07	32 x 3072	ESPNetV2 tiny, width_mult=0.5	CE + Focal boundary loss; distance loss; interior-on-boundary penalty;	0.9605	0.1885	0.8390	0.0956	0.0654
Test-11	64 x 3072	ESPNetV2 tiny, width_mult=0.5	CE + Focal	0.9711	0.2884	0.7587	0.1199	0.1213

Test-12	32 x 3072	ESPNetV2 tiny, width_mult=0.5	CE + Focal	0.9631	0.1885	0.8380	0.0996	0.0624
Test-14	32 x 3072	ESPNetV2 tiny, width_mult=0.5	CE + Focal; distance-weighted Smooth L1 + Distance Gradient Loss, boundary loss, small blob penalty; false positive distance penalty; interior-distance consistency; boundary/interior penalties	0.9631	0.1968	0.8300	0.0926	0.0775
Test-17	32 x 3072	FBSNet	Boundary-aware semantic segmentation	0.9686	0.2227	0.8260	0.0778	0.0962
Test-18	64 x 3072	FBSNet	Boundary-aware semantic segmentation	0.9733	0.2548	0.8158	0.0949	0.0892

The table does not include every trained model. Instead, it contains the configurations that were most relevant for interpreting the results. The full raw evaluation reports were kept as project artefacts and were used to select the runs shown in this table.

The ESPNetV2 results show that the best model depends on the criteria used. Test-11 with 64 x 3072 input size achieved the best ESPNetV2 pixel-level IoU, but it did not achieve the best object-counting behaviour. In contrast, Test-07 and Test-12 with the input size of 32 x 3072 achieved the lowest component count error and the highest results for exact count accuracy among the ESPNetV2 models.

The FBSNet experiment achieved strong results, especially for IoU. The 64 px FBSNet run reached a nut IoU of 0.9733, which is the highest of all the runs shown in the table. However, the object-counting results of this model were not the best, and the ONNX runtime measured on the laptop was significantly lower than other 32 px models. This model's runtime was also measured with the script on Qcify AIR computer and its average runtime per image was 4 ms, which is also a significantly worse result than all other models including the 64 px ESPNetV2 models. The 32 px FBSNet run had a better count MAE than the 64 px FBSNet run, but it still did not outperform the best ESPNetV2 32 px models for object counting. The FBSNet model was also much smaller with 118,524 parameters. This model's size was reduced gradually. The reason for that is the runtime constraints, before when the model had more parameters, its runtime was very high, so for that reason it was optimized.

Overall, the results suggest that Test-11 is the strongest ESPNetV2 candidate from a pixel-level segmentation perspective, while Test-07, Test-12 and Test-14 are more interesting from a practical 32 px deployment perspective. Test-14 is especially useful to include since it combines good segmentation quality, low count error and high ONNX FPS.

4.4. Runtime Results

Runtime was evaluated because the model is intended to run in a real-time industrial machine. The evaluation scripts measure the PyTorch and ONNX performance, including average image latency and FPS. The laptop ONNX benchmarks are useful for comparing models under the same conditions, but they do not fully represent the production machine.

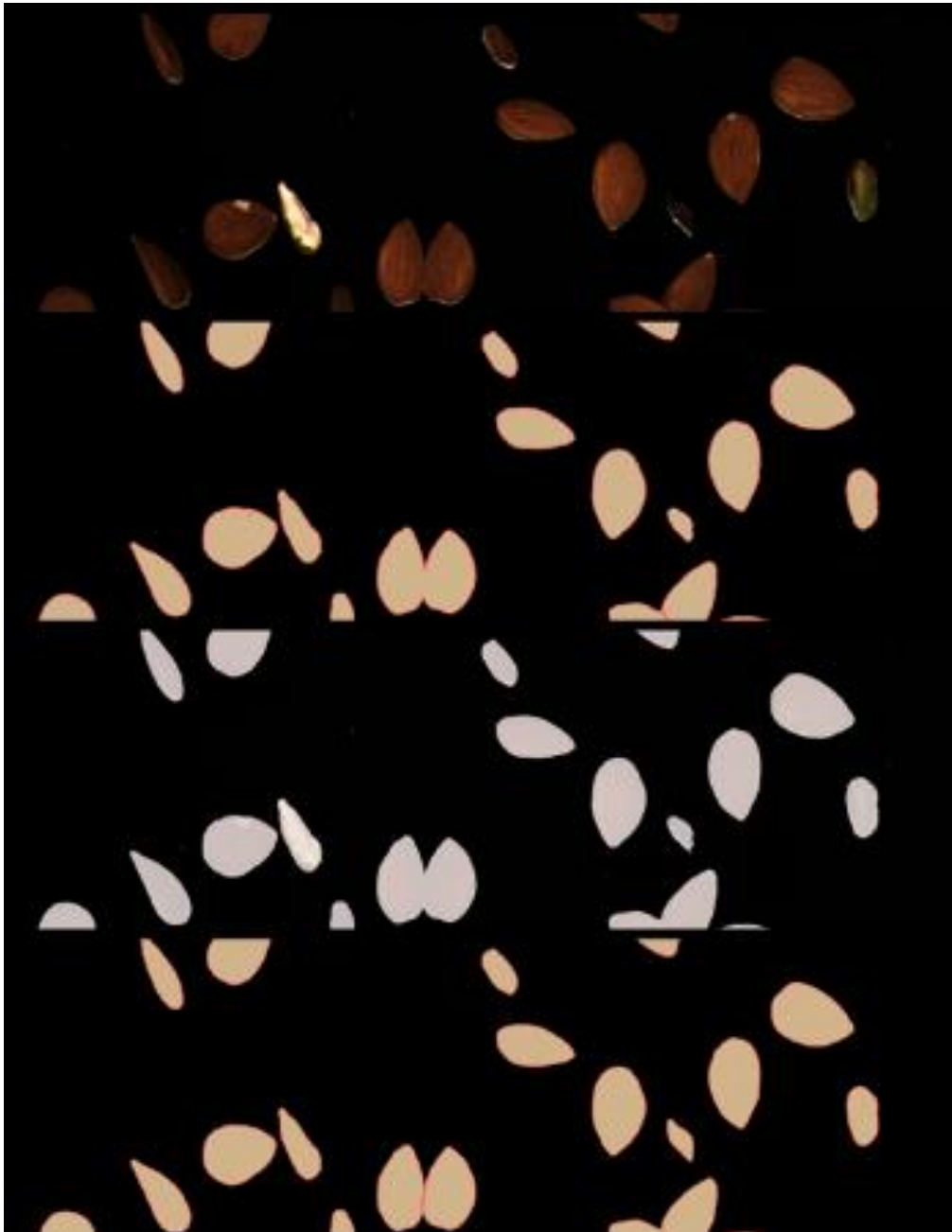
The runtime measurements on the AIR computer showed that on average, a 32 px model took around 1 ms to 2 ms per image, while a 64 px model took around 2 ms to 4 ms per image. The full AIR processing pipeline takes approximately 15-20 ms, including image capture, segmentation, classification, post-processing and sorting impulse. This means that the differences between the inference difference between these two input sizes is relevant, but it should be interpreted in the context of the complete processing pipeline. The 32 px input remains the safer runtime choice, while 64 px can be justified only if the improvement in segmentation quality is significantly better.

4.5. Visual Evaluation

Visual evaluation was used together with quantitative metrics. It was important to do that because some errors are difficult to understand from metrics alone. Examples include holes inside the almond masks caused by predicting boundaries in the almond interior, excessive cuts, merged neighbouring almonds and over-segmentation.

The visualisation script generated panels that contained the input image, predicted class mask, prediction overlay and ground-truth mask. It also supported an option to stitch the sliced images belonging to the same original image back into a larger view. This made it easier to see how the segmentation worked, since the sliced images were of small height, so it was difficult to see the predictions with an eye.

Examples:



This example shows an earlier ESPNetV2 configuration trained with weighted Cross-Entropy and Dice Loss. It is included for comparison, because it shows why visual inspection was necessary: even when the overall mask quality appears to be acceptable, the separation between closely positioned almonds and the placement of boundaries can still be unreliable.

IoU: 0.9614
Count MAE: 0.2082
Merge rate: 0.1026
Split rate: 0.0670

Test-07, 32 px





This example represents the model with the best object-counting amongst all ESPNetV2 runs. The goal of including these images is to show that even though the model does not have the highest IoU, it performs well due to tendency to preserve the correct number of connected components.

IoU: 0.9605
Count MAE: 0.1885
Merge rate: 0.0956
Split rate: 0.0654

Test-11, 64 px





This example shows the ESPNetV2 model with the highest IoU. The 64 px input preserves more spatial detail, which is helpful for the model to produce cleaner masks. However, this does not mean that the model automatically has the best object-counting performance. This example is shown as a comparison to the previous one, because it is clear to see that this model produces cleaner masks (it has less boundary predictions over interior masks across the whole testing dataset).

IoU: 0.9711
Count MAE: 0.2884
Merge rate: 0.1199
Split rate: 0.1213

Test-14, 32 px

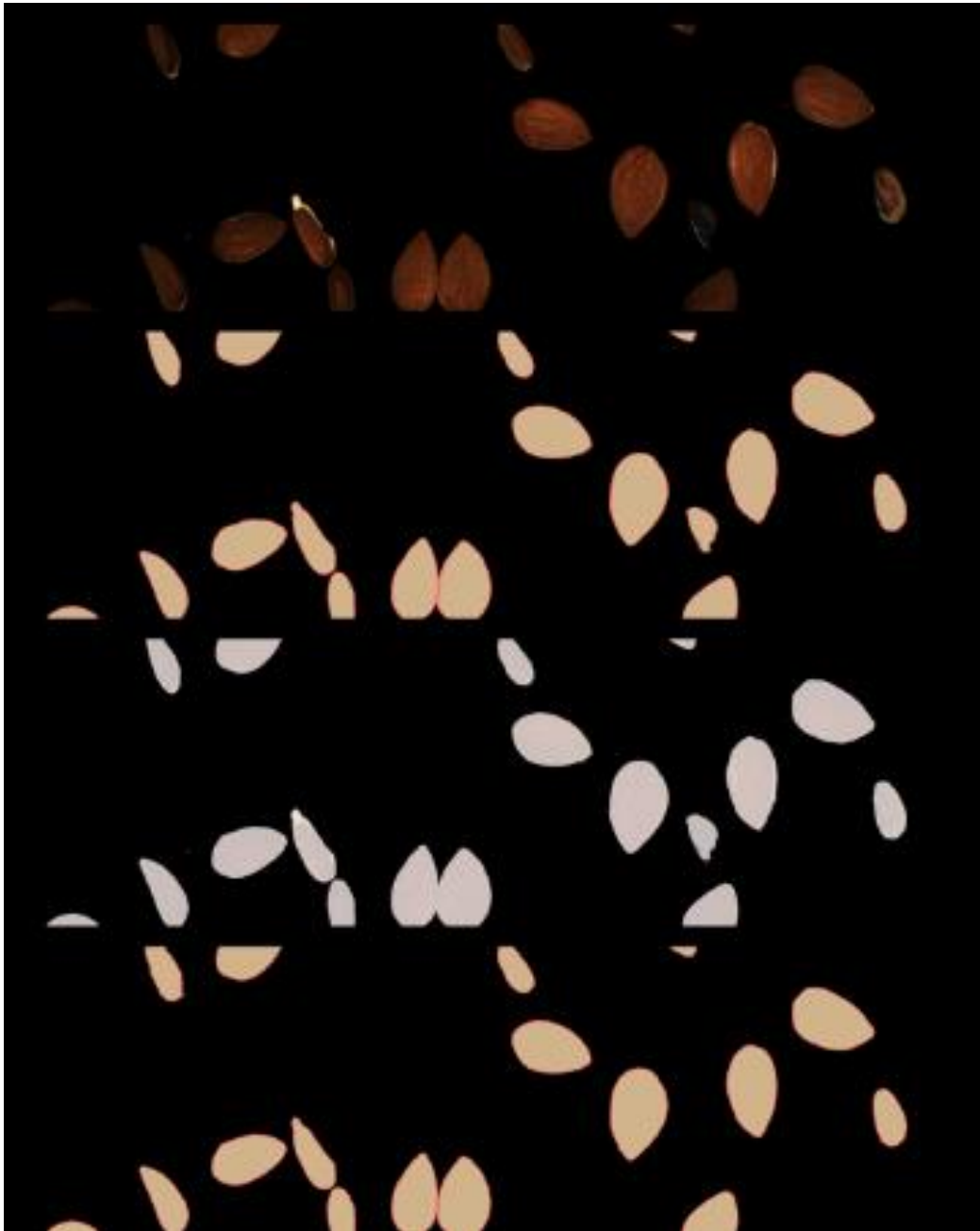




This example shows a balanced 32 px ESPNetV2 configuration with additional penalty terms. It is included because it combines good segmentation quality, low merge rate and fast runtime. Predicted masks are not as clean as the ones from other candidates, however this model remains one of the most relevant candidates for practical use.

IoU: 0.9631
Count MAE: 0.1968
Merge rate: 0.0926
Split rate: 0.0775

Test-18, 64 px





This example shows the additional FBSNet experiment. The model achieved strong IoU, and the masks are visually clean across all images from the test dataset.

IoU: 0.9734
Count MAE: 0.2548
Merge rate: 0.0949
Split rate: 0.0892

Overall, the visual examples confirm that the choice of the best model cannot be based on a single metric. Some models produce cleaner masks and have higher IoU, while others perform better in terms of connected component counting. The visual inspection also helps identify issues that are not fully captured by the metrics, such as merged almonds, boundary prediction in the centre of almond regions or small false positives.

4.6. Discussion of Results

The evaluation showed that the best model depends on the criteria used for comparison. Pixel-level metrics, such as IoU, are relevant for measuring general mask quality. However, they do not fully describe declustering performance. A model can create clean masks but still merge objects. For this reason, it is important to look at both object-level metrics and generated visuals.

The results also showed differences between 32 px and 64 px inputs. In general, the larger inputs had better potential to produce cleaner masks, because their higher input height preserved more spatial information. However, this did not always lead to better object-counting behaviour. The 32 px models remained very competitive, especially because of their better runtime.

Among all the ESPNetV2 runs Test-11 achieved the strongest pixel-level segmentation result, while Test-07 and Test-12 performed better in object-counting metrics. Test-14 was interesting because it combined good segmentation quality, low merge rate and good runtime performance.

The FBSNet experiment showed that a different, lightweight model architecture could also produce strong segmentation results. However, because of limited runs on this configuration, it should be interpreted only as an additional comparison, not as a main outcome of the project. It could perhaps be a good starting point for future experiments.

Overall, the evaluation suggests that the most suitable model candidate should be selected based on a balance between segmentation quality, declustering behaviour and runtime performance. For deployment-oriented use, a strong 32 px ESPNetV2 model is a good direction. If the priority is only pixel-level segmentation quality, 64 px models could be an interesting choice, but their runtime and object-counting precision must still be considered.

5. Conclusion

This project investigated possible ways to improve declustering behaviour in the Qcify AIR vision pipeline. The assignment was approached as a research and development task, which means that the goal was not to guarantee a production-ready solution, but to explore different technical approaches and evaluate which of them showed the most potential.

The starting point was a semantic segmentation pipeline where closely positioned almonds could be predicted as one object. This is a limitation for declustering, because the model does not automatically distinguish between separate object instances when they belong to the same class. Earlier declustering attempts also showed that stronger separation can introduce a different problem: the model may remove too much useful information from almond regions which can lead to misclassification issues.

During the project, several modifications were investigated. The binary segmentation target was extended to a three-class representation. This gave the model more explicit information about object regions and structure where the neighbouring almonds should be separated. Additional tests also included distance maps, boundary head, various loss functions and penalty terms that were aimed at reducing specific failure cases.

The evaluation showed that choosing the best model cannot be done by looking at a single metric. Pixel-level metrics are good for measuring general mask quality, but they do not fully describe the full picture. Other evaluation methods such as object-level and visualisation inspection are also needed, because the model can have a good IoU score but still merge objects together or create unwanted boundaries inside the almond interior mask.

The results identified a couple of promising directions. The 64 px models generally provided strong pixel-level segmentation quality, while some of the 32 px models showed better object counting behaviour and remained closer to the expected runtime. The additional FBSNet approach also showed that alternative lightweight architectures can give promising results. However, more experiments would be needed before making conclusions about this architecture.

Overall, the project provided a structured comparison of boundary-aware semantic segmentation approaches for AIR declustering. It did not fully solve the merging problem, but it clarified which configurations are worth further investigation and which trade-offs should be considered. The most important trade-off is between segmentation quality, object-level separation and runtime performance.

5.1. Future Work

As this project was mainly research-oriented, several directions remain open for future work. Further work could include more experiments with FBSNet or other lightweight architectures. The initial FBSNet results looked promising, but only a limited number of runs were performed. More training configurations would be needed to make a fair comparison with ESPNetV2.

It would be interesting to investigate a post-processing step further. Since the output of the model is still semantic segmentation, additional post-processing may be helpful in converting predicted masks into more reliable separated object regions. This could include connected-component filtering, small-object removal, boundary-based splitting or combining distance maps with watershed-like separation methods.

Finally, boundary size could also be investigated further. All the models were trained with the boundary thickness of 8 pixels. It would be interesting to investigate different sizes, both smaller and larger boundary sizes.

REFERENCE LIST

Azad, R., Heidary, M., Yilmaz, K., Hüttemann, M., Karimijafarbigloo, S., Wu, Y., Schmeink, A., & Merhof, D. (2023). *Loss Functions in the Era of Semantic Segmentation: A Survey and Outlook*. arXiv.

<https://arxiv.org/abs/2312.05391>

DigitalOcean. (n.d.). *Panoptic segmentation: Unifying semantic and instance segmentation*.

<https://www.digitalocean.com/community/tutorials/panoptic-segmentation>

Encord. (n.d.). *Segment Anything Model 2 (SAM 2) & SA-V dataset from Meta AI explained*.

<https://encord.com/blog/segment-anything-model-2-sam-2/>

Gao, G., Xu, G., Li, J., Yu, Y., Lu, H., & Yang, J. (2021). *FBSNet: A Fast Bilateral Symmetrical Network for Real-Time Semantic Segmentation*. arXiv.

<https://arxiv.org/abs/2109.00699>

Gao, G. (n.d.). *FBSNet: A Fast Bilateral Symmetrical Network for Real-Time Semantic Segmentation*.

<https://guangweigao.github.io/TMM22/FBSNet.html>

GeeksforGeeks. (n.d.). *What are different evaluation metrics used to evaluate image segmentation models?*

<https://www.geeksforgeeks.org/computer-vision/what-are-different-evaluation-metrics-used-to-evaluate-image-segmentation-models/>

Hamdaan, M. (2021). *Multi-class semantic segmentation with U-Net & PyTorch*. Medium.

<https://medium.com/@mhamdaan/multi-class-semantic-segmentation-with-u-net-pytorch-ee81a66bba89>

Jadon, S. (n.d.). *Semantic Segmentation Loss Functions*. GitHub.

<https://github.com/shruti-jadon/Semantic-Segmentation-Loss-Functions>

Lightly. (n.d.). *Instance segmentation in practice: Techniques, tools, and applications*.

<https://www.lightly.ai/blog/instance-segmentation>

Mehta, S., Rastegari, M., Shapiro, L., & Hajishirzi, H. (2019). *ESPNetv2: A Light-weight, Power Efficient, and General Purpose Convolutional Neural Network*. arXiv.

<https://arxiv.org/abs/1811.11431>

Pratiher, D. (n.d.). *Understanding Loss Functions for Deep Learning Segmentation Models*. Medium.

<https://medium.com/@devanshipratiher/understanding-loss-functions-for-deep-learning-segmentation-models-30187836b30a>

Ultralytics. (n.d.). *Segment Anything Model (SAM)*. Ultralytics documentation.

<https://docs.ultralytics.com/models/sam>

XU-GITHUB-curry. (n.d.). *FBSNet*. GitHub.

<https://github.com/XU-GITHUB-curry/FBSNet>

[Author unknown]. (n.d.). *Distance-map-based segmentation source*.

<https://d-nb.info/1346554471/34>

[Author unknown]. (2025). *ScienceDirect article on segmentation / distance-map-related computer vision method*. ScienceDirect.

<https://www.sciencedirect.com/science/article/pii/S2643651525000937>

ATTACHMENTS

Attachment A.

Component	Purpose
Convolutional stem	Extracts low-level image features and reduces input resolution
Lightweight residual blocks	Extract features efficiently while keeping the model compact
Decoder upsampling blocks	Restore spatial resolution for dense pixel-wise prediction
Spatial detail path	Preserves fine local information from early feature maps
Spatial attention	Highlights relevant regions such as edges and contact areas
Three-class output	Predicts background, nut boundary and almond interior

Attachment B.

Methods	Input Size	Backbone	Parameters (M) ↓	FLOPs (G)↓	Speed (FPS)↑	mIoU (%)↑
SegNet [27]	640 × 360	VGG16	29.50	286.0	17	57.0
ENet [5]	512 × 1024	No	0.36	3.8	135	58.3
SQNet [41]	1024 × 2048	SqueezeNet	-	270.0	17	59.8
ESPNet [6]	512 × 1024	ESPNet	0.36	-	113	60.3
CGNet [17]	360 × 640	No	0.50	6.0	-	64.8
NDNet [42]	1024 × 2048	No	0.50	14.0	40	65.3
ESPNet-v2 [43]	512 × 1024	ESPNet-v2	-	2.7	80	66.2
EDANet [44]	512 × 1024	No	0.68	-	81	67.3
ADSCNet [45]	512 × 1024	No	-	-	77	67.5
ERFNet [22]	512 × 1024	No	2.10	-	42	68.0
Fast-SCNN [13]	1024 × 2048	No	1.11	-	123	68.0
BiseNet [9]	768 × 1536	Xception39	5.80	14.8	106	68.4
BCPNet [46]	512 × 1024	No	0.61	-	250	68.4
ICNet [7]	1024 × 2048	PSPNet50	26.50	28.3	30	69.5
DABNet [26]	1024 × 2048	No	0.76	10.5	28	70.1
LEDNet [24]	512 × 1024	No	0.94	-	40	70.6
EdgeNet [47]	512 × 1024	-	-	-	31	71.0
DFANet [11]	1024 × 1024	Xception	7.80	3.4	100	71.3
FDDWNet [48]	512 × 1024	No	0.80	-	60	71.5
MSCFNet [2]	512 × 1024	No	1.15	17.1	50	71.9
BiseNet-v2 [10]	512 × 1024	No	3.40	21.2	156	72.6
FBSNet (ours)	512 × 1024	No	0.62	9.7	90	70.9

This table was used as a background material during the research phase. It compares several lightweight segmentation architectures based on parameters, FLOPs, FPS, and mIoU reported in external sources. The values were not measured on the Qcify AIR almond dataset, so they were only used to understand general architecture trade-offs and to support the selection of efficient models such as ESPNetV2 and FBSNet.